

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**Proposal for Integrated Course: Advanced C Programming with Generative AI****To: The Board of Directors / Pro Vice-Chancellor****Date: February 12, 2026**

Subject: This proposal seeks approval for a new integrated course, **Advanced C Programming with Generative AI**, for the second semester (A.Y. 2025-2026), which merges the existing **Advanced C Programming with Copilot (B25CI0201)** and **Advanced C Programming with Copilot Lab (B25CI0202)** components. The course is designed to build upon the foundations of Introduction to C Programming (B25CI102) by focusing on programming fundamentals, memory management, use of Generative AI and automation.

1. EXECUTIVE SUMMARY AND RATIONALE

The integration of theory and laboratory components into the **Advanced C Programming with Generative AI** course is a strategic necessity to evolve with modern pedagogical and industry standards. Since the foundational theory of C programming was comprehensively covered in the Introduction to C Programming (ICP) course during the first semester, this integrated model shifts the focus from passive lecturing to active, application-based mastery.

Key Implementation Mandates:

- **Integrated Credits:** 1-1-1 (Lecture-Tutorial-Practice) for a total of 3 credits.
- **Core Philosophy:** Labs serve as the primary assessment engine, replacing traditional written exams with continuous, program-execution-based evaluation.
- **Theory Assessment:** No written Internal Assessments (IA) and Semester End Examination (SEE) will be conducted for the theory component of the course.
- **Moodle Integration:** All course assessments will be conducted exclusively using the Moodle platform with autograded functionality.
- **Pedagogical Shift:** Since the foundational theory of C programming was comprehensively covered in the Introduction to C Programming (ICP) course during the first semester, this integrated model shifts the focus from traditional lecturing to activity-based instruction facilitated via differentiated learning levels.
- **Course Assessment Weightage:**

Component	Internal Assessment (CIE)	Semester End Exam (SEE)	Total Marks
Theory	20 Marks (MCQs)	25 Marks (MCQs)	45
Lab	25 Marks (Programs)	25 Marks (Programs)	50
Mini-Project	5 Marks (CIE Component)	--	5
Total Weight	50 Marks	50 Marks	100

By integrating theory and lab components:

- **Synchronized Learning:** Students immediately apply advanced concepts like pointers and dynamic memory management in a controlled practice environment, reinforcing learning through immediate feedback.
- **Differentiated Learning Pathways:** Integration allows for a tiered pedagogical approach (Levels 1, 2, and 3), enabling students to progress through the curriculum at a pace that matches their proficiency, with mastery training.
- **Streamlined Assessment:** Under this integrated model, labs serve as the primary assessment engine. Continuous assessment of program execution ensures that students are evaluated on their ability to solve real-world problems rather than their ability to memorize syntax for a written exam.
- **Operational Efficiency:** This model streamlines the curriculum by removing redundant IA exams and replacing them with a continuous, activity-based pedagogy that tracks student progress in real-time.
- **AI-Enhanced Workflow:** The inclusion of Generative AI reflects modern industry standards, allowing students to use AI for enhancing the coding experience through the principles of scaffolding while focusing on high-level logic (critical thinking) and problem-solving.

2. COURSE GENERAL INFORMATION

Course Title	Advanced C Programming with Generative AI				Course Type		HC	
Course Code	B25CI0201	Credits	3		Class/Lab		II Semester	
	LTP	Credits	Contact Hours	Work Load	Total Number of Classes Per Semester		Assessment Weightage	
	Lecture	1	1	1				
	Tutorial	1	2	2				
	Practice	1	2	2	Theory	Practical	CIE	SEE
	Total	3	5	5	42	28	50%	50%

3. COURSE OVERVIEW:

This integrated course is designed to transition students from basic syntax to advanced application development using Advanced C Programming and modern AI-assisted tools. The curriculum emphasizes modular programming and Test-Driven Development, moving beyond simple logic to focus on memory efficiency through pointers and dynamic memory allocation. Students will engage in an activity-based pedagogy organized into three levels of increasing complexity, culminating in a mini-project where they leverage Generative AI to design functional applications through iterative prompt engineering. By

blending core systems-level programming with contemporary AI productivity tools, students will develop the technical rigor and modern workflow required for complex software development.

4. COURSE OBJECTIVE(S):

The objectives of this course are to enable students to:

- **Implement Modular and Testable Code:** Master the creation of modular, reusable code structures using both call-by-value and call-by-reference functions, while adhering to the principles of clean coding and Test-Driven Development.
- **Manage Complex Data:** Develop expertise in organizing and processing complex data by applying modular programming techniques to Arrays of Structures and complex data collections.
- **Memory Management:** Gain a practical understanding of pointers and dynamic memory allocation, allowing for the creation of efficient, flexible, and memory-aware C applications.
- **Handling Text and Processing Files:** Understanding text-based data processing through user-defined string logic and file handling systems for persistent data storage.
- **Integrate AI-Assisted Development:** Foster proficiency in using Generative AI as a collaborative tool to assist in code analysis, iterative design, and the construction of modular mini-projects through effective prompt engineering.
- **Achieve Progressive Mastery:** Progress through a tiered learning structure (Level 1 to Level 3) designed to transition students from foundational execution to mastery-level programming through intensive practice and continuous assessment.

5. COURSE OUTCOMES (COs):

After the completion of the course, the student will be able to:

CO#	Course Outcomes	POs	PSOs
CO1	Develop modular and reusable code using functions (call by value and call by reference), arrays and structures using illustrative programs and test-driven development.	1-5,9, 11	1-3
CO2	Apply modular programming to manage complex data collections using Arrays of Structures through illustrative programs and test-driven development.	1-3, 5	1-3
CO3	Develop modular and reusable code using pointers and dynamic memory allocation using illustrative programs and test-driven development.	1-3	1-3
CO4	Develop modular and reusable code utilizing Strings by implementing user-defined logic for processing text-based data through illustrative programs and test-driven development.	1-3	2

CO5	Develop a modular application for file handling in C using illustrative programs and test-driven development.	1-3, 5, 9	1-3
CO6	Leverage generative AI to design and build a mini-project application, utilizing iterative prompt engineering and code analysis.	1-5,9, 11	1-3

6. BLOOM'S LEVEL OF THE COURSE OUTCOMES

CO#	Bloom's Level					
	Remember (L1)	Understand (L2)	Apply (L3)	Analyze (L4)	Evaluate (L5)	Create (L6)
CO1			✓			
CO2				✓		
CO3			✓			
CO4			✓			
CO5		✓				
CO6						✓

7. COURSE ARTICULATION MATRIX:

CO# / POs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	2	2	2										2	
CO2	2	2	2										2	
CO3	3	3	3									1		
CO4	2	2	3									2	2	
CO5	3	3	3										2	2
CO6	1	1	1		3							2	2	3

Note: 1-Low, 2-Medium, 3-High

8. COURSE CONTENT THEORY

Content
1. C Fundamentals: Quick review of basic C programming concepts (arrays, structures, array of structures and functions). Function prototypes, definition, and scope (local vs. global). Parameter Passing Mechanisms: Call by value and call by reference through modular programming approach. Error handling.

2. **Array of Structures:** Passing arrays to functions as parameters, operators to access structure members, passing structures to functions as parameters, operators to access structure members, passing array of structures to functions as parameters.
3. **Pointers:** Understanding pointers and Dynamic Memory Management, using malloc, calloc, realloc, and free, pointers and arrays, handling dangling pointers and preventing memory leaks, self-referential structures, singly linked lists as an application of pointers.
4. **Text Processing:** Reading and manipulating strings using user-defined functions and pointers.
5. **File handling:** File handling functions for various operations: Opening a File: fopen(), Reading from a File: fread(), fgets(), Writing to a File: fwrite(), fprintf(), Closing a File: fclose(), Error Checking: feof(), ferror(), Different modes of operations in these (r,w,a, binary).
6. **Mini-Project using AI tools and Prompt Engineering utilizing above discussed concepts.**

TEXT BOOKS:

- B.W. Kernighan & D.M. Ritchie, "C Programming Language", 2nd Edition, PRENTICE HALL SOFTWARE SERIES, 2005.
- Herbert Schildt, "C: The Complete Reference", 4th edition, TATA McGraw Hill, 2000.
- B.S. Anami, S.A. Angadi and S. S. Manvi, "Computer Concepts and C Programming: A Holistic Approach", second edition, PHI, 2008.
- Yashavant Kanetkar, "Let us C", BPB, 19th edition, 2022

REFERENCE BOOKS:

- Balaguruswamy, "Programming in ANSI C", 4th edition, TATA MCGRAW Hill, 2008.
- Jon Bentley, "Programming Pearls", 2nd edition, PEARSON, 2002.

JOURNALS/MAGAZINES:

- <https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=6294> (IEEE Journal/Magazine on IT Professional)
- <https://ieeexplore.ieee.org/document/1267572> (IEEE Computing in Science and Engineering)

SWAYAMNPTEL/MOOCs

- https://online.courses.nptel.ac.in/noc20_cs06/preview (Problem Solving through Programming in C)
- <https://www.edx.org/course/c-programming-getting-started> (C Programming Getting started)
- <https://www.coursera.org/specializations/c-programming> (Introduction to C programming)

9. PEDAGOGY

The course follows a tiered, activity-based structure:

- **Level 1 (Foundational):** 10 Programs focusing on core execution.
- **Level 2 (Intermediate):** 10 Programs.
- **Level 3 (Mastery):** 10 Programs.

Operational Flow:

- **Instruction:** Faculty complete 2 programs weekly in the regular classroom post 1st week of commencement of the course. All levels to be completed by the faculty in the regular classroom.
- **Progression:** A class test is conducted after Level 1 to transition students to Level 2 (non-credit diagnostic).
- **Support:** Slow-learners (students who are unable to complete level 1 in first 6 weeks) receive online scaffolding for Level 1 by Dr. Ramaprasad H C; advanced learners attend daily online mastery training from Dr. Srinivasan for Levels 2 and 3.
- Level 2 and level 3 would be conducted as-is in regular classroom by faculty post the first 6 weeks for all learners.

10. ASSESSMENT AND EVALUATION RUBRICS

Traditional IA1 and IA2 exams are removed in favor of **Continuous Internal Evaluation**.

A. Continuous Internal Evaluation (CIE) - 50 Marks

Component	Breakdown	Total
Theory (MCQs)	10 Marks (CIE1) + 10 Marks (CIE2)	20 Marks
Mini-Project	Based on level of complexity (L1 to L3)	5 Marks
Lab (Programs)	15 Marks (CIE1) + 10 Marks (CIE2)	25 Marks
Total		50 Marks

CIE Assessment Protocol

The course utilizes a continuous, activity-based evaluation model facilitated through the Moodle platform, which eliminates traditional IA exams in favor of weekly proficiency tracking. The assessment follows these specifications:

- **Practical Evaluation:** Students are required to execute the two programs mastered during the previous week in the subsequent lab session for formal evaluation on the Moodle platform.
- **Integrated Theory Testing:** For each program executed, a set of 10 Multiple Choice Questions (MCQs) is dynamically generated from a pool of standardized questions to assess underlying conceptual knowledge on the Moodle platform.
- **Component Mapping:**

- **Theory Component:** Performance in the MCQ sets constitutes the marks for the Theory component of the CIE.
- **Lab Component:** Program execution and logic implementation constitute the marks for the Practical/Lab component of the CIE.
- **Scaling:** Marks are aggregated and scaled as follows:

Theory Component:

Assessment	# Assessments	# MCQs Per Assessment	Total Raw Marks	Scaled Marks
CIE1	5	10	50	10
CIE2	5	10	50	10
Total				20

Lab Component:

Assessment	# Programs	Marks per Program	Total Marks
CIE1	5	3	15
CIE2	5	2	10
Total			25

Mini-Project Component:

Category	Marks (Max 5)
Level 1 + Mini Project	3 / 5
Level 2 + Mini Project	4 / 5
Level 3 + Mini Project	5 / 5

- The course utilizes a continuous, activity-based evaluation model facilitated exclusively through the **Moodle platform**, which eliminates traditional IA exams in favor of weekly proficiency tracking. **Consequently, no traditional manual evaluation sheets or physical lab records will be utilized for the lab component of this course.**

B. Semester End Examination (SEE) - 50 Marks

The SEE consists of two distinct components designed to evaluate both theoretical comprehension and practical implementation.

Theory Component (25 Marks)

The theory examination evaluates conceptual understanding through a digital assessment platform.

Theory	# MCQs	Total Marks	Duration
	25	25	60 minutes

- **Platform:** Theory SEE will be conducted for the entire cohort simultaneously using Moodle and is autograded.

Lab Component (25 Marks)

The practical examination focuses on the execution of programs.

Lab	# Programs	Marks per Program	Total Marks	Duration
	5	5	25	3 hours

- **Examination Format:** Each student will be required to execute a total of five programs, assigned through a randomized selection process on the Moodle platform.
- **Program Distribution:** To ensure comprehensive evaluation across proficiency levels, the assessment will consist of three randomized programs from Level 1 (Foundational), one randomized program from Level 2 (Intermediate), and one randomized program from Level 3 (Mastery).

11. Minimum Proficiency Requirement: To successfully clear the course, a student must satisfy the following cumulative and component-specific thresholds:

- **Aggregate Passing Mark:** A student must achieve a cumulative minimum of 40 marks out of 100 across all components (CIE + SEE).
- **SEE Threshold:** Out of the 40 Aggregate Passing Marks, a minimum of 20 marks must be scored specifically within the SEE components (Theory SEE + Lab SEE).
- **SEE Component Minimums:**
 - **Theory SEE:** Minimum 10 marks out of 25.
 - **Lab SEE:** Successful execution of a minimum of 2 programs.

12. Level 1 Programs:

Program No.	Program Details	CO
1	Write a modular C program to read n float values from the user, store them in an array and find the maximum value and its index. (array, passing arrays to functions, for loop, if statement) Function prototype: <pre>void input(int n, float arr[n]); int find_max_index(int n, float arr[n]); void output(float arr[], int max_index);</pre>	1
2	Write a C program to compare areas of three rectangles and find the largest using functions and structure. (else-if ladder) <pre>typedef struct { char name[50]; float length; float width; float area; } Rectangle; Function prototype: Rectangle input(); float calculate_area(Rectangle rect); void compare_areas(Rectangle r1, Rectangle r2, Rectangle r3);</pre>	1
3	Write a C program using array of structures and functions to: - Accept the number of rectangles. - For each rectangle, accept its length and width. - Calculate and display the area of each rectangle. - Determine and display the rectangle with the largest area. (array of structures, for loop, if statement) <pre>typedef struct { float length; float width; float area; } Rectangle; Function prototype: void input(int n, Rectangle rects[n]);</pre>	2

	<pre>void calculate_area(int n, Rectangle rects[n]); int findLargestArea(int n, Rectangle rects[n]); void output(int largestIndex, Rectangle rects[]);</pre>	
4	Write a modular C program to create a structure Flight with flight_number, destination, and available_seats. Create an array of 4 flights. Write a program that takes a destination name from the user and checks if a flight is available to that location, displaying the flight number if found. Function prototype: <pre>void readFlights(int n, Flight_t f[]); void searchByDestination(int n, Flight_t f[], char searchDest[]);</pre>	2
5	Write a program to swap two integer arrays. Function prototype: <pre>void read_array(int n, int arr[]); void print_array(int n, int arr[]); void swap_array(int n, int a[n], int b[n]);</pre>	3
6	Develop a modular C program to manage an integer array using dynamic memory allocation. You must implement specific functions to allocate, initialize, display, and safely deallocate memory. This task ensures proficiency in preventing memory leaks and managing dangling pointers. Function prototype: <pre>int* create_array(int n); void initialize_array(int *arr, int n); void print_array(int *arr, int n); void delete_array(int **arr);</pre>	3
7	Write a C program to compare two strings using user-defined functions. The program should return: • 0 if both strings are equal • 1 if the first string is lexicographically greater • -1 if the second string is lexicographically greater Function prototype: <pre>void inputStrings(char str1[], char str2[]); int compareStrings(char str1[], char str2[]); void output(int result);</pre>	4

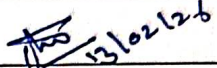
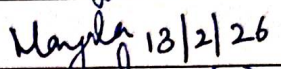
8	Write a C program to concatenate two strings using user-defined functions. The program should accept two strings from the user and pass them to a function that performs the concatenation. (strings, while/do while) Function prototype: <pre>void input(char *str); void concatenate_strings(char *str1, char *str2); void display(char *str);</pre>	4
9	Write a modular C program to perform the following tasks using file handling functions: - Read the details of n students from the user and store them in an array of structures. - Write the array of structures to a text (ASCII) file using fprintf(). - Read the data back from the ASCII file into a second array using fscanf() and display it. - Display the array of structures. Function prototype: <pre>void inputStudents(Student students[], int n); void writeToTextFile(Student students[], int n, const char *filename); int readFromTextFile(Student students[], const char *filename); void printStudents(Student students[], int n);</pre>	5
10	Write a modular C program to perform the following tasks using file handling functions: - Read the details of n students from the user and store them in an array of structures - Write the array of structures to a binary file using fwrite(). - Read the binary file into a second array of structures using fread(). - Display the array of structures. Function prototype: <pre>void inputStudents(Student students[], int n); void writeToBinaryFile(Student students[], int n, const char *filename); int readFromBinaryFile(Student students[], const char *filename); void printStudents(Student students[], int n);</pre>	5

13. Recommendation for Approval:

It is recommended that the Board reviews the proposed integrated course Advanced C Programming with Generative AI with aforementioned protocols and assessment rubrics.

Submitted for your review and approval.

Approval signatures:

Director, SoCSE, Group 1	Dr. Ashwinkumar U Motagi	 13/02/26
Director, SoCSE, Group 2	Dr. Sarvamangala D R	 13/2/26
Director, SoCSE, Group 3	Dr. G Parthasarathy	 N.P. Mel 14/2/26
Pro Vice-Chancellor	Dr. K S Narayanaswamy	 16.02.26